

Companion Document: update_L.R

q(L) — Variational Update for Loading Matrix

Andrew Walther

2026-03-25

Table of contents

0.1	Purpose and Context	2
0.2	Mathematical Background	2
0.2.1	Model	2
0.2.2	CAVI Update for L	2
0.3	Function Reference	3
0.3.1	compute_R_k	3
0.3.2	update_L_k	4
0.3.3	update_L_all	5
0.4	Dual-Source Precision and Signal	6
0.5	Test Suite	7
0.5.1	T1: Mathematical Identity Checks (6 tests)	7
0.5.2	T2: Survival Term Cancellation (3 tests)	7
0.5.3	T3: Signal Recovery (2 tests)	7
0.5.4	T4: Multi-Factor K=5 (3 tests)	8
0.5.5	T5: Null Factor Shrinkage (2 tests)	8
0.5.6	T6: Precision Scaling (3 tests)	8
0.5.7	T7: Numerical Stability (4 tests)	8
0.5.8	T8: compute_R_k and Gauss-Seidel (3 tests)	9
0.5.9	T9: V2.R Consistency (2 tests)	9
0.6	Demos	9
0.6.1	Demo 1: Anatomy of A_L and B_L	9
0.6.2	Demo 2: Signal Recovery K=1	9
0.6.3	Demo 3: Genomics-only vs. Genomics + Survival	10
0.6.4	Demo 4: Error-in-Variables — Factor Uncertainty	10
0.6.5	Demo 5: Multi-Factor Recovery K=5	10
0.7	Key Mathematical Properties	10
0.7.1	Vector vs. Scalar EBNM	10
0.7.2	Second Moment Identity	11
0.7.3	A_floor for Stability	11
0.7.4	Gauss-Seidel vs. Jacobi	11
0.7.5	tau_j Cancellation Does NOT Apply Here	11
0.8	Connections to Other Modules	11

0.9	Common Pitfalls	12
0.9.1	Forgetting $EL^2 \neq EL^2$	12
0.9.2	A_{gen} is Scalar, A_L is a Vector	12
0.9.3	$z_{\text{no_k}}$ Computed Inline	12
0.9.4	Sourcing Order for <code>compute_R_k</code>	12
0.9.5	Prior Family Selection	12
0.9.6	Cox Weights w Must Be Non-Negative	13

0.1 Purpose and Context

`update_L.R` implements the CAVI (Coordinate Ascent Variational Inference) update for the loading matrix L ($n \times K$) in a supervised Bayesian matrix factorization model. The model jointly explains:

- **Genomics:** $Y = L F'$ ($n \times p$ matrix of expression data)
- **Survival:** Cox PH hazard $h(t|i) = h(t) \exp(L)$

Because L appears in **both** the genomics likelihood and the survival likelihood, its posterior update receives signal from two sources simultaneously. This dual-source structure is the defining feature of this module and what distinguishes $q(L)$ from $q(F)$ (which only sees genomics).

The update is a **vector EBNM** (Empirical Bayes Normal Means) problem: each sample i has its own effective precision $A_L[i]$ and effective signal $B_L[i]$, so we solve n EBNM problems simultaneously via the `ebnm` package.

0.2 Mathematical Background

0.2.1 Model

$$Y_{ij} = \sum_k L_{ik} F_{jk} + E_{ij}, \quad E_{ij} \sim N(0, 1/\tau_j)$$

$$\log h(t|i) = \log h_0(t) + \sum_k L_{ik} \beta_k$$

0.2.2 CAVI Update for L

Taking the ELBO derivative with respect to $q(l_{\cdot k})$ (the k -th loading column), the optimal variational distribution is Gaussian with:

Precision (A_L):

$$A_L[i] = A_{\text{gen}} + A_{\text{surv}}[i]$$

$$A_{\text{gen}} = \sum_j (\tau_j * E[f_{\cdot jk}^2]) \quad \# \text{ scalar, same for all } i$$

$$A_{\text{surv}}[i] = w[i] * E[\beta_k^2] \quad \# \text{ n-vector, sample-specific}$$

where $w[i] = W_{ii}$ is the diagonal of the Cox partial-likelihood Hessian (sample-specific curvature).

Signal (B_L):

```

B_L[i]      = B_gen[i] + B_surv[i]
B_gen[i]    = sum_j( tau_j * E[f_{jk}] * R_k[i,j] )    # genomics contribution
B_surv[i]   = w[i] * z_no_k[i] * E[beta_k]             # survival contribution

```

where $z_no_k[i] = z[i] - \eta_no_k[i]$ is the partial Cox residual after removing factor k 's contribution.

EBNM arguments:

```

x_L[i] = B_L[i] / A_L[i]      # n-vector: effective observation
s_L[i] = 1 / sqrt(A_L[i])    # n-vector: effective noise SD

```

The ebnm posterior gives: $- \text{mean_L}[i] = E_q[l_{\{ik\}}] - \text{sd_L}[i] = SD_q(l_{\{ik\}}) - EL2[i,k] = \text{Var}_q(l_{\{ik\}}) + \text{mean_L}[i]^2$ (second moment, not squared mean)

The second moment EL2 is used in downstream updates for $q(F)$ and $q(\cdot)$ to correct for uncertainty in L (error-in-variables correction).

0.3 Function Reference

0.3.1 compute_R_k

```
compute_R_k(Y, EL, EF, k)
```

Purpose: Compute the partial residual matrix with factor k removed.

Arguments:

Argument	Type	Dimension	Description
Y	matrix	$n \times p$	Observed genomics data
EL	matrix	$n \times K$	Current posterior means $E_q[L]$
EF	matrix	$p \times K$	Current posterior means $E_q[F]$
k	integer	scalar	Factor index to remove (1-based)

Returns: $n \times p$ matrix

Formula:

```

R_k = Y - EL %*% t(EF) + outer(EL[,k], EF[,k])
      = Y - sum_{j != k} EL[,j] * EF[,j] '

```

This removes all factors except k from Y , leaving only the residual that factor k should explain. The `outer(EL[,k], EF[,k])` term adds back factor k 's contribution that was over-subtracted by the full `EL %*% t(EF)`.

Shared dependency: `compute_R_k` is defined in `update_L.R` but is also used by `update_F.R`. When sourcing, `update_L.R` must be sourced first.

0.3.2 update_L_k

```
update_L_k(Tau, EF_k, EF2_k, w, EBeta_k, EBeta2_k, R_k, z_no_k,
           prior_family = "point_normal", A_floor = 1e-10)
```

Purpose: Vector EBNM update for the k -th loading column $l_{\{.k\}}$ (n observations).

Arguments:

Argument	Type	Dimension	Description
Tau	vector	p	Posterior mean of noise precisions $E_q[\tau_{.j}]$
EF_k	vector	p	Posterior mean of factor k : $E_q[f_{\{.k\}}]$
EF2_k	vector	p	Posterior second moment of factor k : $E_q[f_{\{.k\}}^2]$
w	vector	n	Cox Hessian diagonal (survival weights $W_{.ii}$)
EBeta_k	scalar	1	Posterior mean of β_k : $E_q[\beta_k]$
EBeta2_k	scalar	1	Posterior second moment of β_k : $E_q[\beta_k^2]$
R_k	matrix	$n \times p$	Partial residual from <code>compute_R_k</code>
z_no_k	vector	n	Partial Cox residual with factor k removed
prior_family	string	—	ebnm prior family (default: "point_normal")
A_floor	numeric	scalar	Minimum precision floor (default: $1e-10$)

Returns: Named list

Element	Dimension	Description
mean	n -vector	Posterior mean $E_q[l_{\{.k\}}]$
second	n -vector	Posterior second moment $E_q[l_{\{.k\}}^2]$
sd	n -vector	Posterior SD $\sqrt{\text{Var}_q(l_{\{.k\}})}$
A	n -vector	Total precision $A_L = A_{\text{gen}} + A_{\text{surv}}$
B	n -vector	Total signal $B_L = B_{\text{gen}} + B_{\text{surv}}$
B_gen	n -vector	Genomics contribution to signal
B_surv	n -vector	Survival contribution to signal
x	n -vector	EBNM input: $x_L = B_L / A_L$
s	n -vector	EBNM input: $s_L = 1 / \sqrt{A_L}$
ebnm_result	list	Full <code>ebnm()</code> output object

Internal computation:

```

# Precision
A_gen  <- sum(Tau * EF2_k)           # scalar
A_surv <- w * EBeta2_k               # n-vector
A_L    <- pmax(A_gen + A_surv, A_floor) # n-vector, floored

# Signal
B_gen  <- R_k %*% (Tau * EF_k)       # n-vector (matrix-vector product)
B_surv <- w * z_no_k * EBeta_k       # n-vector
B_L    <- B_gen + B_surv

# EBNM arguments
x_L    <- B_L / A_L
s_L    <- 1 / sqrt(A_L)

# EBNM call
fit     <- ebnm(x = x_L, s = s_L, prior_family = prior_family)

```

0.3.3 update_L_all

```

update_L_all(Y, EL, EL2, EF, EF2, Tau, w, z, EBeta, EBeta2,
             prior_family = "point_normal", A_floor = 1e-10)

```

Purpose: Gauss-Seidel sweep over all K factors, updating L column by column.

Arguments:

Argument	Type	Dimension	Description
Y	matrix	$n \times p$	Observed genomics data
EL	matrix	$n \times K$	Posterior means $E_q[L]$ (read; updated copy returned)
EL2	matrix	$n \times K$	Posterior second moments $E_q[L^2]$
EF	matrix	$p \times K$	Posterior means $E_q[F]$
EF2	matrix	$p \times K$	Posterior second moments $E_q[F^2]$
Tau	vector	p	Posterior mean noise precisions $E_q[\tau_j]$
w	vector	n	Cox Hessian diagonal W_{ii}
z	vector	n	Full Cox working residual z
EBeta	vector	K	Posterior means $E_q[\beta]$
EBeta2	vector	K	Posterior second moments $E_q[\beta^2]$
prior_family	string	—	ebnm prior family (default: "point_normal")
A_floor	numeric	scalar	Minimum precision floor (default: $1e-10$)

Returns: Named list

Element	Dimension	Description
EL	$n \times K$	Updated posterior means
EL2	$n \times K$	Updated posterior second moments
details	K-list	Per-factor output from <code>update_L_k</code> for each k

Gauss-Seidel mechanism:

Each factor k is updated using the *most recent* values of all other factors. After updating column k , `EL_curr[,k]` and `EL2_curr[,k]` are immediately overwritten, so factor $k+1$ sees the already-updated factor k . This improves convergence rate compared to a simultaneous (Jacobi) update.

z_no_k computed inline (no dependency on `update_beta.R`):

```
eta_no_k <- EL_curr %*% EBeta - EL_curr[,k] * EBeta[k]
z_no_k   <- z - eta_no_k
```

This computes the Cox linear predictor without factor k , then takes the residual.

0.4 Dual-Source Precision and Signal

The following table summarizes the dual-source structure — the central feature of the $q(L)$ update:

Component	Source	Dimension	Formula	Notes
<code>A_gen</code>	Genomics	scalar	$\text{sum}(\text{Tau} * \text{EF2}_k)$	Same for all samples
<code>A_surv[i]</code>	Survival	n-vector	$w[i] * \text{EBeta2}_k$	Sample-specific via Cox weights
<code>A_L[i]</code>	Combined	n-vector	$\text{pmax}(\text{A_gen} + \text{A_surv}[i], \text{A_floor})$	Floored for numerical stability
<code>B_gen[i]</code>	Genomics	n-vector	$(\text{R}_k \% \% (\text{Tau} * \text{EF}_k))[i]$	Weighted inner product with residual
<code>B_surv[i]</code>	Survival	n-vector	$w[i] * z_no_k[i] * \text{EBeta}_k$	Survival residual signal
<code>B_L[i]</code>	Combined	n-vector	$\text{B_gen}[i] + \text{B_surv}[i]$	Total signal
<code>x_L[i]</code>	EBNM	n-vector	$\text{B_L}[i] / \text{A_L}[i]$	Effective observation
<code>s_L[i]</code>	EBNM	n-vector	$1 / \text{sqrt}(\text{A_L}[i])$	Effective noise SD

Why A_L varies across samples: The Cox Hessian diagonal $w[i] = W_{ii}$ reflects each patient's contribution to the partial likelihood curvature. Patients at high risk (near their event time) have larger $w[i]$, so their survival signal pulls harder on their loading estimate. This is in contrast to A_gen , which applies equally to all samples because the genomics noise precision tau_j depends on genes, not patients.

Special cases:

Condition	Effect on A_L	Effect on B_L
$w = 0$ (all)	A_L is constant (= A_gen, scalar)	B_surv = 0
EBeta_k = 0	A_surv = 0 (pure genomics precision)	B_surv = 0
EBeta2_k = 0	A_surv = 0	B_surv = 0
$\tau = 0, w = 0$	A_L hits A_floor	B = 0 $\rightarrow$ EL = 0

0.5 Test Suite

File: tests/test_update_L.R **Total tests:** 28 across 9 groups **Status:** All passing (105/105 overall test suite)

0.5.1 T1: Mathematical Identity Checks (6 tests)

Verifies the core formulas hold numerically to tolerance.

Test	What is checked
T1.1	$A_L[i] == \sum(\tau * EF2_k) + w[i] * EBeta2_k$ for each i
T1.2	$B_{gen}[i] == (R_k \%* \% (\tau * EF_k))[i]$ for each i
T1.3	$B_{surv}[i] == w[i] * z_{no_k}[i] * EBeta_k$ for each i
T1.4	x is an n-vector (length n)
T1.5	$s[i] == 1 / \sqrt{A_L[i]}$ for each i
T1.6	$second[i] == sd[i]^2 + mean[i]^2$ (second moment identity)

0.5.2 T2: Survival Term Cancellation (3 tests)

Test	Condition	Expected outcome
T2.1	$w = 0$	B_surv = 0; pure genomics signal
T2.2	EBeta_k = 0	B_surv = 0
T2.3	$w = 0$	A_L is constant across all i (scalar broadcast)

0.5.3 T3: Signal Recovery (2 tests)

Test	Setup	Expected
T3.1	Simulate $Y = L_{true} * F' + \text{noise}$; $K=1$	$\text{cor}(EL, L_{true}) > 0.7$

Test	Setup	Expected
T3.2	Same but with survival signal added	Survival tightens posterior; recovery at least as good

0.5.4 T4: Multi-Factor K=5 (3 tests)

Test	What is checked
T4.1	Output EL has dimensions $n \times K$, EL2 has dimensions $n \times K$
T4.2	All $EL2[i,k] \geq EL[i,k]^2$ (second moment $\geq$ squared mean)
T4.3	All outputs finite (no NaN or Inf)

0.5.5 T5: Null Factor Shrinkage (2 tests)

Test	Setup	Expected
T5.1	Noisy R_k and z_{no_k} (no real signal)	Posterior mean near zero (shrinkage)
T5.2	$EF_k = 0, w = 0$	A hits floor, $B = 0, EL = 0$

0.5.6 T6: Precision Scaling (3 tests)

Test	What is checked
T6.1	Larger $EF2 \rightarrow$ larger $A_{gen} \rightarrow$ more shrinkage (smaller posterior mean)
T6.2	Larger $EBeta2 \rightarrow$ larger $A_{surv} \rightarrow$ more shrinkage
T6.3	Combined genomics + survival: $ B_{combined} > B_{gen} $ when signals align

0.5.7 T7: Numerical Stability (4 tests)

Test	Condition	Expected
T7.1	$\tau = 0, w = 0$	A hits A_{floor} ; no NaN/Inf in any output
T7.2	$\tau = 1e8$ (extreme)	No overflow; outputs finite
T7.3	$n = 1, p = 1$ (edge case)	Function runs without error
T7.4	Custom $A_{floor} = 1e-5$	$\min(A_L) \geq 1e-5$

0.5.8 T8: compute_R_k and Gauss-Seidel (3 tests)

Test	What is checked
T8.1	compute_R_k excludes factor k ($R_k + \text{outer}(EL[,k], EF[,k]) - Y - \text{rest}$)
T8.2	compute_R_k returns matrix of dimension $n \times p$
T8.3	update_L_all uses Gauss-Seidel: $EL[,1]$ after update differs from initial before $k=2$'s R_k is computed

0.5.9 T9: V2.R Consistency (2 tests)

Test	What is checked
T9.1	update_L_k output matches equivalent lines 310–321 of Supervised_Bayesian_MF_V2.R
T9.2	compute_R_k output matches equivalent lines 290–291 of Supervised_Bayesian_MF_V2.R

0.6 Demos

File: demos/demo_update_L.R **Structure:** 5 scenarios; narrative output only (no PASS/FAIL)

0.6.1 Demo 1: Anatomy of A_L and B_L

Constructs a small synthetic problem ($n=50$, $p=100$, $K=1$) and displays the dual-source breakdown: - Prints A_{gen} (scalar) - Prints $\text{range}(A_{\text{surv}})$ (n -vector range due to Cox weight variation) - Prints $\text{range}(A_L)$ (combined) - Prints $\text{range}(B_{\text{gen}})$, $\text{range}(B_{\text{surv}})$, $\text{range}(B_L)$

Teaching point: Even though A_{gen} is the same for every sample, A_{surv} introduces per-sample variability. The effective observation $x_L = B_L / A_L$ is therefore sample-specific.

0.6.2 Demo 2: Signal Recovery $K=1$

Simulates a rank-1 model: - $Y = L_{\text{true}} * t(F_{\text{true}}) + \text{noise}$ - Calls update_L_k with true F as EF - Reports $\text{cor}(\text{result\$mean}, L_{\text{true}})$

Teaching point: Even a single update recovers the true loading direction with correlation > 0.7 when signal-to-noise is reasonable.

0.6.3 Demo 3: Genomics-only vs. Genomics + Survival

Compares two runs: - Run 1: $w = \text{rep}(0, n)$ (survival disabled) - Run 2: $w > 0$ (survival enabled, $\text{beta}_k \neq 0$)

Reports posterior SDs under each regime.

Teaching point: Survival information tightens the posterior (smaller posterior SDs) because $A_{\text{surv}} > 0$ increases total precision. The dual-source update is strictly more informative than genomics alone.

0.6.4 Demo 4: Error-in-Variables — Factor Uncertainty

Compares two settings: - EF2_low : factor second moments close to EF^2 (low uncertainty) - EF2_high : factor second moments much larger than EF^2 (high uncertainty)

Reports A_{gen} and posterior shrinkage under each.

Teaching point: When factors are uncertain ($\text{EF2} \gg \text{EF}^2$), A_{gen} is inflated, increasing shrinkage on loadings. This is the error-in-variables correction: the model is appropriately conservative when factors are poorly estimated.

0.6.5 Demo 5: Multi-Factor Recovery $K=5$

Simulates a rank-5 model: - SVD initialization for EL and EF - Calls `update_L_all` - Reports column-wise correlations between `result$EL` and `L_true`

Teaching point: The Gauss-Seidel loop recovers all 5 factors simultaneously. Correlations are typically > 0.6 after a single sweep, improving with CAVI iterations.

0.7 Key Mathematical Properties

0.7.1 Vector vs. Scalar EBNM

Unlike `update_beta.R` (which solves a *scalar* EBNM — one observation per factor) and `update_F.R` (which solves p scalar EBNM problems — one per gene), `update_L.R` solves a *vector* EBNM with n observations simultaneously. The `ebnm` package accepts vector inputs (x, s) and fits a shared prior across all n entries.

Module	EBNM type	x dimension	s dimension	Why
<code>update_beta</code>	Scalar	1	1	One beta_k scalar per factor
<code>update_L</code>	Vector (n)	n	n	L varies across samples; A_{surv} is per-sample
<code>update_F</code>	Vector (p)	p	p	F varies across genes; A_F is per-gene

0.7.2 Second Moment Identity

For any posterior q :

$$\begin{aligned}\text{EL2}[i,k] &= \text{Var}_q(l_{\{ik\}}) + (\text{E}_q[l_{\{ik\}}])^2 \\ &= \text{sd}[i]^2 + \text{mean}[i]^2\end{aligned}$$

This is NOT $\text{mean}[i]^2$. Using mean^2 in place of second is a common error (Correction R3 in the project's error log) that underestimates posterior variance and introduces bias in $q(F)$ and $q()$ updates.

0.7.3 A_{floor} for Stability

When both $\text{Tau} = 0$ and $w = 0$, the precision A_L would be exactly zero, making $s_L = 1/\text{sqrt}(0)$ undefined. The floor $A_{\text{floor}} = 1e-10$ prevents this. When A hits the floor and $B = 0$, `ebnm` returns a degenerate posterior concentrated at 0 — correct behavior for an unidentified factor.

0.7.4 Gauss-Seidel vs. Jacobi

The Gauss-Seidel update (each column updated sequentially with immediate propagation) is one of the V2 improvements (label A1 in `PROJECT_STATUS.md`). It typically converges faster than the simultaneous Jacobi update used in V1 because each update uses strictly more current information.

0.7.5 tau_j Cancellation Does NOT Apply Here

In `update_F.R`, the per-gene precision tau_j cancels out of the effective observation x_j (though it remains in s_j). This cancellation does NOT occur in `update_L.R` because the genomics signal B_{gen} involves a sum over all genes j , weighted by tau_j . Different genes contribute different amounts of signal to each patient's loading estimate.

0.8 Connections to Other Modules

Module / File	Relationship
<code>code/update_F.R</code>	Shares <code>compute_R_k</code> (defined here, must source <code>update_L.R</code> first)
<code>code/update_beta.R</code>	Provides <code>z_no_k</code> pattern (replicated inline in <code>update_L_all</code>)
<code>code/update_tau.R</code>	Consumes EL2 (second moments computed here)
<code>code/Supervised_Bayesian_MF_V2.R</code>	Reference implementation; lines 290–291 (R_k), 310–321 (L_k update)
<code>derivations/qL/qL_update_deriv.R</code>	Full derivation of dual-source ELBO gradient and EBNM reduction

Module / File	Relationship
tests/test_update_L.R	28-test suite; run with Rscript tests/run_tests.R
demos/demo_update_L.R	5 narrative demos
code/SupervisedMF_Context.md	Project-wide quick-reference for AI/code context

Source order dependency: When using update_F.R, always source update_L.R first:

```
source("code/update_L.R") # defines compute_R_k
source("code/update_F.R") # uses compute_R_k
```

0.9 Common Pitfalls

0.9.1 Forgetting $EL2 \neq EL^2$

The second moment $EL2[i,k] = sd^2 + mean^2$ is **not** $mean^2$. Downstream modules (update_tau, update_F) use $EL2$ explicitly. Passing EL^2 instead will underestimate the variance correction (Correction R3).

0.9.2 A_{gen} is Scalar, A_L is a Vector

$A_{gen} = \sum(Tau * EF2_k)$ is a single scalar. It becomes part of the n-vector A_L only after adding A_{surv} . Broadcasting in R handles this automatically ($A_{gen} + A_{surv}$ where A_{surv} is a vector), but it is worth being explicit: A_L always has length n.

0.9.3 z_{no_k} Computed Inline

update_L_all does NOT call compute_z_no_k from update_beta.R. The equivalent computation is done inline to avoid a circular source dependency. Both implementations are mathematically equivalent.

0.9.4 Sourcing Order for compute_R_k

compute_R_k lives in update_L.R. If update_F.R is sourced without first sourcing update_L.R, calls to compute_R_k will fail with “could not find function”. Always source update_L.R first.

0.9.5 Prior Family Selection

The default prior_family = “point_normal” imposes sparsity (exact zeros for inactive loadings). For dense loadings, consider “normal”. The choice affects both the posterior means and the estimated prior parameters — see the ebnm package documentation for alternatives.

0.9.6 Cox Weights w Must Be Non-Negative

The Cox Hessian diagonal $w[i] = W_{ii} \geq 0$ always. Negative values would make A_{surv} negative, potentially driving A_L below zero (before the floor). The floor catches this, but negative weights indicate a bug in the Cox weight computation upstream.

This document corresponds to the implementation in `code/update_L.R` and its tests in `tests/test_update_L.R`. For the full mathematical derivation, see `derivations/qL/qL_update_derivation.pdf`.